

MLIR for AI / NPU Compiler

Lecture 20 Capstone

Mini NPU Compiler Design Spec 작성

학습 목표

- 20 강 전체에서 배운 MLIR 개념을 하나의 NPU compiler architecture 로 통합한다.
- StableHLO/TOSA 입력에서 npu_runtime command buffer 까지 이어지는 lowering boundary 를 설계한다.
- legal op set, dialect contract, transform schedule, bufferization/memory plan, runtime ABI 를 문서화한다.
- FileCheck, negative test, cost model, demo script 까지 포함한 검증 가능한 compiler design spec 을 작성한다.

1. Capstone 의 목적

20 강은 앞선 19 강의 개념을 하나의 설계 산출물로 통합하는 강의입니다. 학생은 단순히 MLIR operation 을 읽는 수준을 넘어서, target NPU 의 제약을 compiler IR, pass pipeline, runtime ABI, verification plan 으로 표현해야 합니다.

Capstone 에서 중요한 질문

1. 어떤 입력 IR 을 compiler 의 semantic contract 로 삼을 것인가?
2. 어떤 op 가 target NPU 에서 legal 이고, 어떤 op 는 decompose/fallback 되어야 하는가?
3. fusion, layout propagation, quantization 을 어느 abstraction level 에서 수행할 것인가?
4. tile size 와 SRAM/ACCUM footprint 는 어떻게 검증할 것인가?
5. 최종 runtime ABI 는 tensor descriptor 와 command buffer 로 어떻게 표현할 것인가?

Lecture 20 Capstone: Mini NPU Compiler Pipeline



Final artifact: Mini NPU Compiler Design Spec + pseudo MLIR pipeline + tests + demo presentation

2. 최종 산출물 구성

산출물	내용	예제 파일
-----	----	-------

Design Spec	input/legal ops/pipeline/dialect/runtime/test 를 포함한 핵심 문서	02_mini_npu_compiler_design_spec_template.md
Pipeline Spec	pass order, anchors, legality, fallback policy 를 YAML 로 정리	03_pipeline_spec.yaml
Pseudo MLIR	npu_kernel/npu_rt dialect 로 최종 IR 예시 작성	04_npu_dialect_capstone_pseudo.mlir
Schedule	Transform Dialect 또는 schedule YAML 로 tile/fuse/vectorize 표현	05_transform_schedule_capstone.mlir
Runtime ABI	tensor descriptor, command packet, artifact manifest 설계	06_runtime_abi_manifest.yaml
Verification	FileCheck positive/negative test 및 cost model	07_filecheck_capstone_tests.mlir

3. Target NPU 가정

강의용 Capstone 은 특정 상용 NPU 가 아니라 아래와 같은 교육용 target 을 가정합니다. 실제 프로젝트에서는 이 부분이 hardware architecture spec, firmware ABI, compiler capability database 로 대체됩니다.

항목	가정	Compiler 의미
Compute tile	$M \times N \times K = 16 \times 16 \times 32$	vector.contract 또는 linalg.matmul 을 native tile 로 tensorize
Data types	INT8 x INT8 -> INT32, BF16/FP16 optional	requant 또는 epilogue fusion 필요
Memory spaces	DRAM=0, SRAM=1, ACCUM=2	memref memory_space 와 runtime descriptor 에 반영
DMA	explicit async load/store + barrier token	double buffering 설계 가능
Runtime	descriptor table + command buffer	embedded/edge deployment 에 적합

4. Capstone Pass Pipeline

```
builtin.module(
  import-semantic-audit,
  canonicalize,cse,
  decompose-composites,
  legalize-to-npu-graph,
  fuse-epilogues,
  propagate-layout,
  apply-transform-schedule,
  one-shot-bufferize,
  assign-memory-spaces,
  lower-to-npu-kernel,
  emit-runtime-abi
)
```

단계	역할	검증 포인트
Import semantic audit	input IR 의 shape/dtype/layout/operator semantic 을 보존	unsupported op report
Legalization	target NPU legal op set 으로 변환	ConversionTarget / dynamic legality
Fusion/Layout/Quant	traffic 과 kernel launch 수 감소	fusion legality + cost model
Schedule	tile/fuse/vectorize 명시	Transform IR 또는 schedule YAML
Bufferize/Memory	tensor value semantics 를 physical	alias/lifetime/SRAM footprint

	buffer 로 변환	
NPU Dialect	DMA/compute/barrier 를 target contract 로 표현	verifier + negative tests
Runtime ABI	descriptor table 과 command buffer 로 package	ABI version + packet schema

5. Input IR 와 Legal Op Set

StableHLO/TOSA 는 model frontend 와 compiler backend 사이의 semantic input 으로 유용하지만, target NPU 가 모든 operator 를 직접 지원하지는 않습니다. Capstone 에서는 legal op set 을 이름 목록이 아니라 shape/dtype/layout/quantization 조건을 가진 contract 로 작성해야 합니다.

Op	Legal 조건	Lowering	Fallback
matmul	rank-2 or batched rank-3, K static, tile/tail policy	npu.matmul	CPU fallback or split
conv2d	NHWC/HWCF, stride/dilation subset	npu.conv2d or im2col+matmul	fallback
bias add	broadcast axis matches output N/C	fused epilogue	materialize add
GELU/ReLU	supported approximation or LUT	fused epilogue	decompose or fallback
requant	scale/zero-point available, acc width safe	npu.requant	tosa.rescale fallback

6. Custom NPU Dialect Contract

Capstone 의 custom dialect 는 upstream MLIR 에 바로 넣기 위한 완성 구현이 아니라, 어떤 정보가 op operand/result/attribute/type/verifier 에 있어야 하는지 명확히 하는 설계 도구입니다.

```
%c = npu.matmul %a, %b, %acc
{m = 16 : i64, n = 16 : i64, k = 32 : i64,
 lhs_layout = "mk", rhs_layout = "kn", acc_type = "i32"}
: memref<16x32xi8, 1>, memref<32x16xi8, 1>, memref<16x16xi32, 2>
-> memref<16x16xi32, 2>
```

- Operand: runtime data dependency 가 있는 값. 예: tile buffer, accumulator buffer, DMA token.
- Attribute: compile-time schedule/contract. 예: tile size, layout, approximation mode, saturation mode.
- Type: memory/layout/dtype invariant. 예: memref memory_space, quantized element type.
- Verifier: native tile shape, memory space, acc type, barrier ordering 등 local invariant 검사.

7. Schedule 와 Memory Planning

Schedule 은 성능을 결정하는 핵심 산출물입니다. “tile 16x16x32 를 사용한다”는 선언만으로는 충분하지 않습니다. SRAM footprint, double buffering, edge tile, layout packing, epilogue fusion 이 함께 정의되어야 합니다.

SRAM bytes per tile = DB * (M*K*bytes(A) + K*N*bytes(B)) + M*N*bytes(ACC)
For INT8xINT8->INT32 with M=16,N=16,K=32 and double buffering:
DB * (16*32*1 + 32*16*1) + 16*16*4 = 2*(512+512)+1024 = 3072 bytes

Buffer	Memory Space	Shape/DType	Policy
A tile	SRAM	16x32xi8	double buffered
B tile	SRAM	32x16xi8	double buffered
C accumulator	ACCUM	16x16xi32	single buffer or ping-pong
Output tile	SRAM/DRAM	16x16xi8	after requant/epilogue

8. Runtime ABI 와 Command Buffer

Runtime ABI 는 compiler 와 firmware/runtime 사이의 계약입니다. Capstone 에서는 최소한 tensor descriptor layout, command buffer header, packet types, capability/version check 를 정의해야 합니다.

```
struct NpuTensorDesc {
    uint64_t base;
    uint64_t offset;
    uint32_t rank;
    int64_t dims[8];
    int64_t strides[8];
    uint32_t dtype;
    uint32_t layout;
    uint32_t memory_space;
    float  quant_scale;
    int32_t quant_zero_point;
};
```

9. Verification Matrix

Verification Matrix for Mini NPU Compiler

Stage	Positive Test	Negative Test	Metric
Import	StableHLO/TOSA parse	dynamic rank unsupported	shape/dtype coverage
Legalize	all ops legal	illegal GELU path	fallback rate
Fusion/Layout	MLP fused	layout conflict	traffic saving
Schedule	tile fits SRAM	over-capacity tile	cycles / SRAM
Bufferize	no extra copies	alias conflict	copy count
NPU IR	dma-barrier valid	missing await	verifier pass
Runtime ABI	descriptor matches	version mismatch	launch success

10. 평가 Rubric

Category	Weight	Excellent Criteria
Correctness	25	IR contract, verifier invariant, FileCheck/negative tests
Architecture Fit	20	NPU tile, SRAM/ACCUM, DMA/barrier, runtime ABI 반영
Optimization	20	fusion/layout/quant/schedule 선택의 비용 근거
Implementation Plan	15	pass pipeline, boundary, fallback, implementation risk
Validation	10	golden numeric, cost model,

		regression strategy
Presentation	10	10 분 발표의 명확성, tradeoff 설명

11. 제출 형식

- Design Spec: 8-12 pages 또는 equivalent Markdown.
- Pseudo MLIR: before/after IR, npu dialect sample, runtime IR sample.
- Pipeline Spec: pass order 와 legality/fallback policy.
- Verification: 최소 2 개 positive test, 2 개 negative test, cost model output.
- Presentation: 10 분 발표 + 5 분 Q&A.

12. 참고 문서

- MLIR Language Reference: <https://mlir.llvm.org/docs/LangRef/>
- MLIR Pass Management: <https://mlir.llvm.org/docs/PassManagement/>
- MLIR Dialect Conversion: <https://mlir.llvm.org/docs/DialectConversion/>
- MLIR Transform Dialect: <https://mlir.llvm.org/docs/Dialects/Transform/>
- MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>
- MLIR Bufferization: <https://mlir.llvm.org/docs/Bufferization/>
- MLIR LLVM IR Target: <https://mlir.llvm.org/docs/TargetLLVMIR/>
- StableHLO Specification: <https://openxla.org/stablehlo/spec>
- MLIR TOSA Dialect: <https://mlir.llvm.org/docs/Dialects/TOSA/>